

LERNIX

AI-Powered Educational Curriculum Design with IBM Granite 3.3 2B

Project Description

LERNIX is a **Generative AI-powered Curriculum and Learning Management Platform** designed to simplify and personalize the educational planning process for students, educators, and institutions. The platform automates the creation of **structured academic curricula, study plans, quizzes, and learning roadmaps** using **Artificial Intelligence**, significantly reducing the time and effort required for manual curriculum design.

The primary objective of **LERNIX** is to address the challenges faced by educators in designing comprehensive learning pathways and by students in organizing effective study schedules. By leveraging a **locally hosted Large Language Model (LLM)** powered by **IBM Granite through Ollama**, the system generates **customized curricula** based on user inputs such as **subject area, learning goals, skill level, and course duration**. This AI-driven approach enables **fast, scalable, and context-aware content generation** while ensuring **data privacy, security, and reduced operational costs** by avoiding dependence on external cloud-based AI services.

The platform supports **three major user roles: Students, Educators, and Administrators**. **Students** can register, define their learning objectives, and receive **personalized study plans, weekly learning schedules, recommended resources, AI-generated quizzes, and progress tracking tools**. **Educators** can create and manage academic programs, generate **AI-assisted course content**, monitor student performance, and export curricula in multiple formats. **Administrators** oversee user management, content moderation, platform operations, and system analytics through a centralized dashboard.

LERNIX is developed using a modern technology stack consisting of **Flask** as the backend framework, **SQLite** as the database, and **HTML5, CSS3, JavaScript, and Bootstrap** for the frontend. Additional functionalities include **role-based authentication, AI-powered learning assistance, automated quiz generation, roadmap creation, PDF report generation, subscription management, and personalized resource recommendations**.

The platform offers several benefits, including **rapid curriculum generation, personalized learning experiences, continuous progress monitoring, multi-role access management, and professional report exports**. By integrating **Generative AI** with learning management capabilities, **LERNIX** enhances educational efficiency, improves learner engagement, and provides a scalable solution for modern digital education. It serves as a comprehensive platform that bridges the gap between curriculum planning, content delivery, assessment, and student progress tracking within a single intelligent ecosystem.

Scenarios

Scenario 1: AI-Powered Curriculum Generation

A student logs into **LERNIX** and provides details such as the learning topic, skill level, duration, and weekly study hours. The system sends this information to the **IBM Granite Large Language Model through Ollama**, which generates a structured curriculum containing modules, learning objectives, topic-wise roadmaps, study schedules, and recommended resources. The generated curriculum is displayed to the student and can be downloaded as a PDF for future reference.

Scenario 2: Personalized Study Plan Creation

A student selects a curriculum and specifies the number of hours available for study each week. Based on the student's goals and schedule, LERNIX creates a personalized weekly learning plan. The system organizes topics into manageable tasks, recommends learning resources, and provides a structured roadmap to help the student complete the curriculum efficiently within the specified timeframe.

Scenario 3: AI-Generated Quiz and Assessment

After completing a learning module, the student chooses the **Generate Quiz** option. LERNIX analyzes the completed topics and uses the AI model to generate relevant assessment questions. The student attempts the quiz, and the system evaluates the answers, calculates the score, and provides feedback. This helps students assess their understanding and identify areas that require additional study.

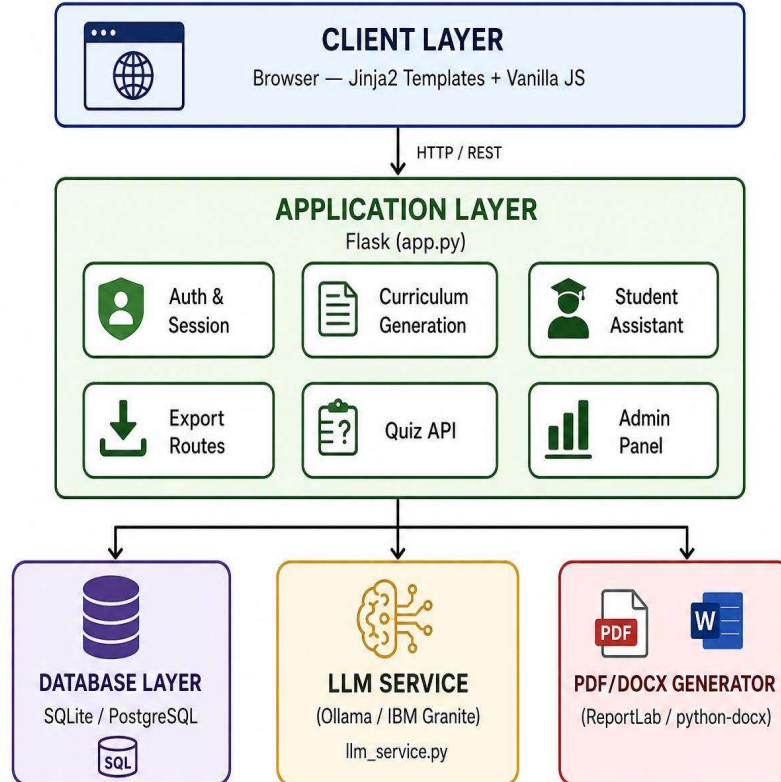
Scenario 4: Progress Tracking and Learning Analytics

As students complete topics, quizzes, and learning activities, LERNIX continuously updates their progress records. The dashboard displays important metrics such as completed modules, pending topics, quiz performance, and overall curriculum completion percentage. Students can monitor their learning journey, stay motivated, and ensure they are progressing toward their educational goals.

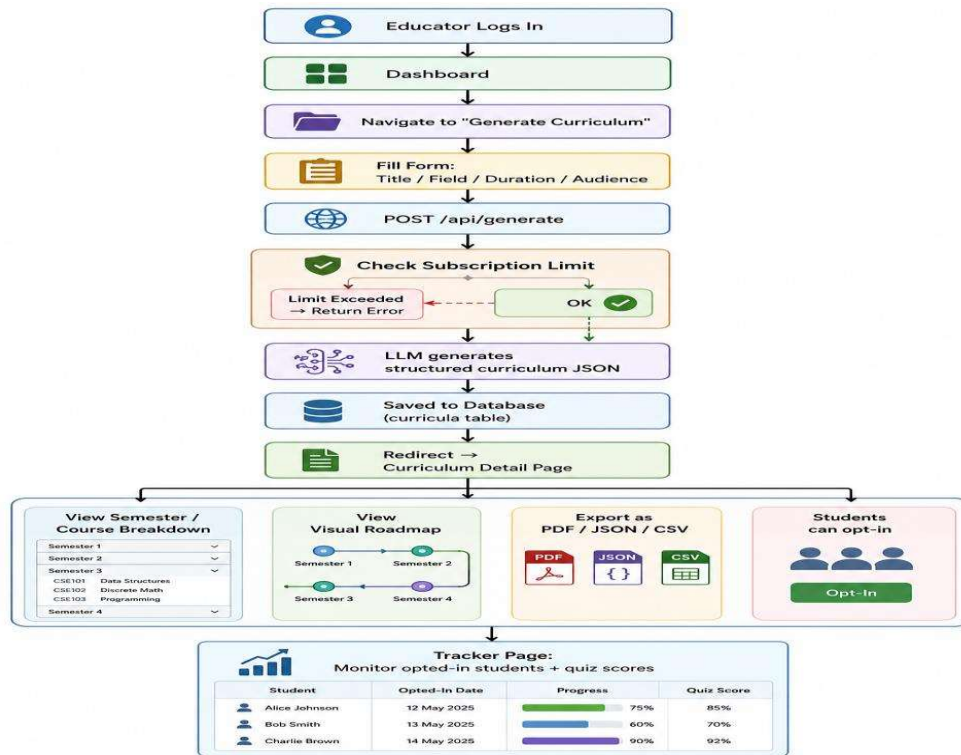
Architecture Overview

LERNIX follows a **three-tier web architecture** that separates the presentation, application, and service layers to ensure scalability, maintainability, and efficient system management. The **client layer** consists of Jinja2-based HTML templates enhanced with JavaScript and Bootstrap to provide an interactive user interface. Users interact with the system through features such as curriculum generation, quiz participation, progress tracking, and roadmap visualization. These interactions are sent as HTTP requests to the Flask-based backend, which processes user actions and returns the appropriate responses.

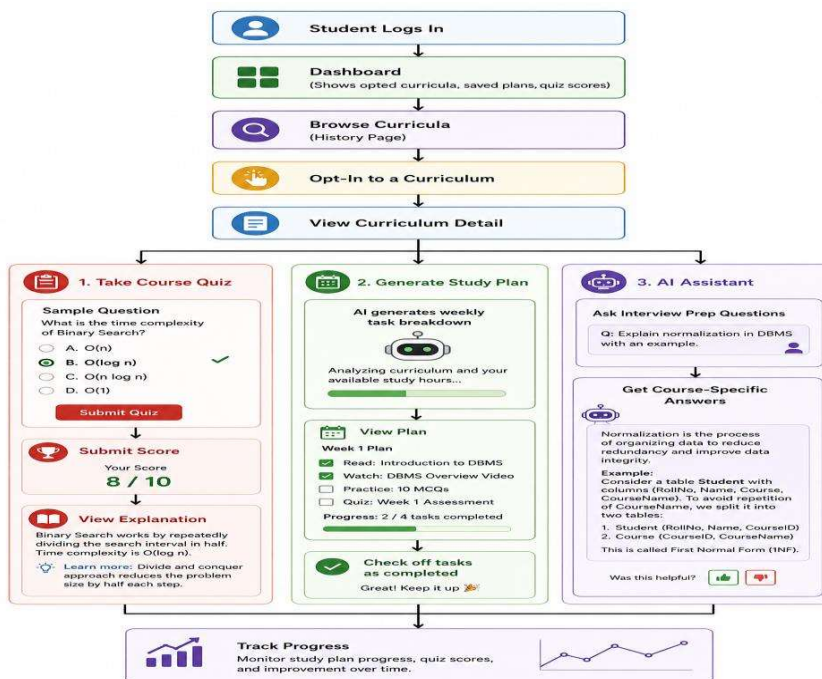
The **application layer (app.py)** serves as the central controller of the system, managing user authentication, session handling, role-based access control, and request routing. Based on the user's action, it communicates with the service layer, which is divided into specialized modules. The **database module** manages SQLite operations and stores user, curriculum, quiz, and progress data. The **LLM service module** interacts with the IBM Granite model through Ollama to generate AI-powered curricula, study plans, quizzes, and learning assistance. The **PDF generation module** creates downloadable curriculum and report documents. This modular architecture allows each component to operate independently, making the system easier to maintain, upgrade, and scale.



Educator Workflow Diagram



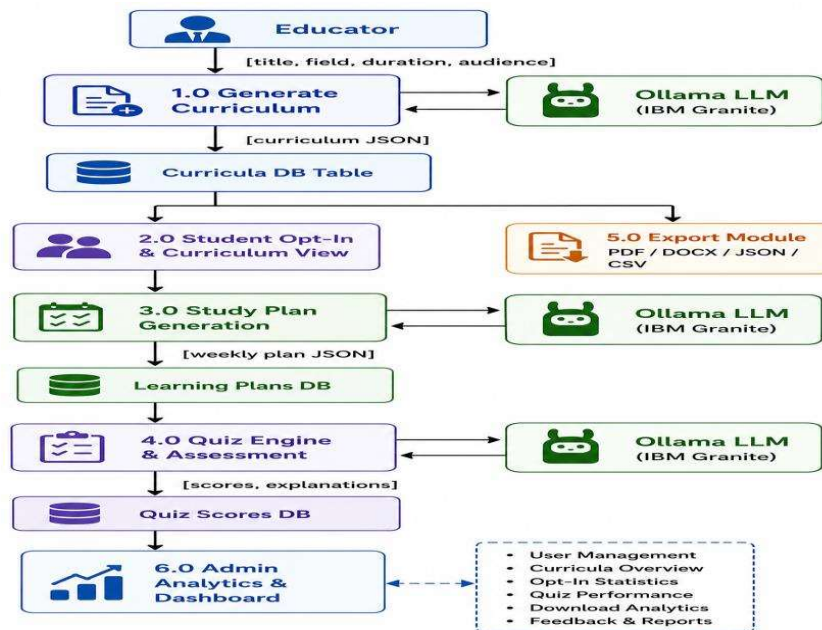
Student Workflow Diagram



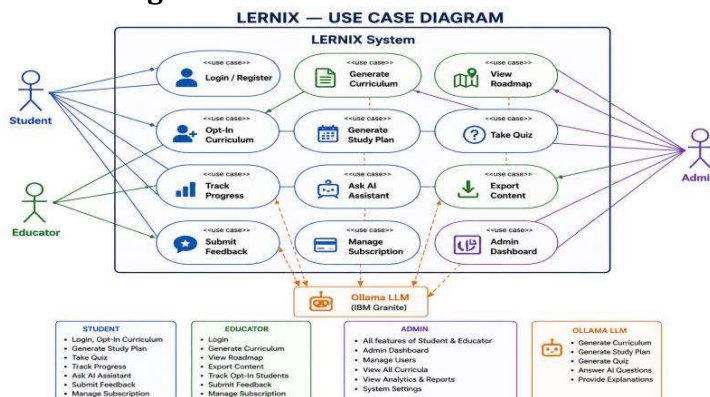
Data Flow Diagram



Level 1 DFD



Use Case Diagram



Technology Stack

Frontend Technologies

- **HTML5** – Provides the structure and layout of web pages.
- **CSS3** – Used for styling, responsive design, and enhancing the user interface.
- **JavaScript** – Enables client-side interactivity, dynamic content updates, and user interactions.
- **Bootstrap** – Provides responsive design, pre-built UI components, and improved user experience.
- **Jinja2 Templates** – Used for server-side rendering of dynamic web pages in Flask.

Backend Technologies

- **Python** – Core programming language used for application development and business logic implementation.
- **Flask** – Lightweight web framework used to handle routing, request processing, authentication, session management, and communication with AI services.
- **Werkzeug Security** – Used for password hashing and secure user authentication.

Database

- **SQLite** – Lightweight relational database used to store:
 - User information
 - Student and administrator details
 - Generated curricula
 - Personalized study plans
 - Quiz and assessment records
 - Progress tracking data
 - Learning resources
 - Curriculum history

Artificial Intelligence

- **Ollama** – Local AI inference platform used to run Large Language Models securely on the local machine.
- **IBM Granite Model** – Large Language Model used for:
 - Curriculum generation
 - Personalized study plan creation
 - Learning roadmap generation
 - AI-powered quiz generation
 - Educational content recommendations
 - Learning assistance and academic guidance
 - Topic explanations and study support

Authentication & Security

- **Flask Session Management** – Used for secure user authentication and session handling.
- **Role-Based Access Control (RBAC)** – Controls access permissions for students, educators, and administrators.
- **Password Hashing** – Ensures secure storage and protection of user credentials.

Report Generation

- **ReportLab** – Used for generating professional PDF curriculum reports.
- **Python File Handling Libraries** – Used for exporting curriculum data and learning reports.

API Communication

- **HTTP Requests** – Enable communication between the frontend, Flask backend, database, and AI processing modules.
- **Ollama API** – Facilitates interaction between the application and the IBM Granite model for AI-generated content.

Development Tools

- **Git & GitHub** – Version control, source code management, and collaborative development.
- **Visual Studio Code (VS Code)** – Primary development environment used for coding, debugging, and project management.

Pre-requisites

Software Requirements

- **Python 3.10+** – Required for developing and running the Flask-based backend application and AI integration modules.
- **Ollama** – Local AI inference platform used to run the IBM Granite Large Language Model for curriculum generation and learning assistance.
- **IBM Granite Model** – Large Language Model used for AI-powered curriculum creation, study plan generation, quizzes, and learning recommendations.
- **SQLite Database** – Used for storing user information, generated curricula, study plans, quizzes, progress records, and application data.
- **Git** – Used for version control, source code management, and project collaboration.
- **Code Editor** – Visual Studio Code, PyCharm, or any preferred Python IDE for development and debugging.
- **Web Browser** – Google Chrome, Mozilla Firefox, Microsoft Edge, or any modern browser for accessing the application.
- **Required Python Libraries** – Flask, ReportLab, Requests, SQLite3, Jinja2, and other project dependencies specified in the requirements file.

Knowledge Prerequisites

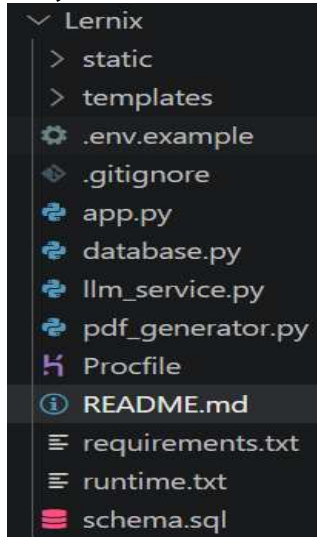
- **Python Programming** – Understanding of functions, object-oriented programming, exception handling, and file management.
- **Flask Framework** – Knowledge of routing, request handling, session management, template rendering, and web application development.
- **HTML5 & CSS3** – Understanding of web page structure, responsive design, and user interface styling.
- **JavaScript** – Knowledge of DOM manipulation, event handling, and dynamic frontend interactions.
- **Database Concepts** – Basic understanding of SQLite, database design, SQL queries, and CRUD operations.
- **Generative AI Concepts** – Understanding of Large Language Models (LLMs), prompt engineering, AI content generation, and Ollama integration.
- **Web Development Fundamentals** – Knowledge of client-server architecture, HTTP requests, authentication, and web application deployment.
- **Educational Domain Knowledge** – Basic understanding of curriculum planning, learning pathways, assessments, and student progress tracking.

Hardware Requirements

- **Processor:** Intel Core i3/i5, AMD Ryzen 3/5, or higher.
- **RAM:** Minimum 4 GB (8 GB recommended for smooth execution of the AI model and application).
- **Storage:** Minimum 5 GB free disk space for project files, database storage, Python dependencies, and Ollama models.
- **Internet Connection:** Required for downloading dependencies, project updates, and deployment activities.
- **Operating System:** Windows 10/11, Linux, or macOS with Python and Ollama support.

Project Workflow

Project Structure



Milestone 1: Environment Setup & Project Configuration

This milestone establishes the development environment by installing Python dependencies, configuring the FastAPI backend, setting up the SQLite database, and configuring Ollama connectivity for AI-powered curriculum generation.

Activity 1.1: Install Dependencies & Setup Environment

Step 1: Create requirements.txt

```
requirements.txt Lernix\requiremen  
1  Flask>=3.0.0  
2  werkzeug>=3.0.0  
3  reportlab>=4.0.0  
4  requests>=2.31.0  
5  gunicorn>=21.2.0  
6  python-dotenv>=1.0.0  
7  python-docx>=1.1.0
```

Step 2: Create Virtual Environment

```
python-m venv venv
```

Activate virtual environment:

Windows

```
venv\Scripts\activate
```

Mac/Linux

source venv/bin/activate

Step 3: Install Dependencies

pip install -r requirements.txt

Activity 1.2: Set Up Local AI Model Using Ollama

Create .env file:

```
.env.example Lernix\.env.example
1  # Copy this file to .env and fill in your values
2  FLASK_APP=app.py
3  FLASK_ENV=production
4  FLASK_DEBUG=0
5  SECRET_KEY=change-this-to-a-long-random-secret-key
6  DATABASE_PATH=lernix.db
7  # Optional: Ollama LLM endpoint
8  OLLAMA_BASE_URL=http://localhost:11434
9
```

Milestone 2: Core Backend Development

This milestone develops the Flask backend and integrates the IBM Granite Large Language Model through Ollama for AI-powered curriculum and study plan generation. The backend handles user authentication, role-based access control, curriculum generation, personalized learning roadmaps, quiz generation, progress tracking, educational resource recommendations, database management using SQLite, and PDF export functionality. It serves as the central processing unit of the system, enabling seamless interaction between users and AI-driven educational services while ensuring efficient, secure, and scalable content generation.

```
def educator_or_admin_required(f):
    from functools import wraps
    @wraps(f)
    def decorated_function(*args, **kwargs):
        if 'user_id' not in session:
            return redirect(url_for('login_page'))
        if session.get('role') not in ['educator', 'admin']:
            flash("Unauthorized. Educator or Admin privileges required.", "danger")
            return redirect(url_for('dashboard_page'))
        return f(*args, **kwargs)
    return decorated_function

# --- Page Controllers ---

@app.route('/')
@app.route('/login')
def login_page():
    if 'user_id' in session:
        return redirect(url_for('dashboard_page'))
    return render_template('login.html')

@app.route('/forgot_password')
def forgot_password_page():
    return render_template('forgot_password.html')

@app.route('/logout')
def logout_action():
    session.clear()
    flash("You have been logged out successfully.", "success")
    return redirect(url_for('login_page'))

@app.route('/dashboard')
@login_required
def dashboard_page():
    user_id = session['user_id']
    user_role = session['role']
```

```

        feedbacks=feedbacks,
        user_details=user_details,
        saved_plans=saved_plans,
        quizzes=quizzes
    )

@app.route('/generate')
@login_required
@educator_or_admin_required
def generate_page():
    user = database.get_user_by_id(session['user_id'])
    return render_template('generate.html', user_details=user)

@app.route('/curriculum/<int:id>')
@login_required
def curriculum_page(id):
    curr = database.get_curriculum(id)
    if not curr:
        flash("Curriculum not found.", "danger")
        return redirect(url_for('dashboard_page'))

    parsed_json = json.loads(curr['raw_json'])
    sum_courses = sum(len(sem.get('courses', [])) for sem in parsed_json.get('semesters', []))
    total_credits = sum(sum(int(c.get('credits', 0)) for c in sem.get('courses', [])) for sem in parsed_json.get('semesters', []))

    # Check if student is opted in
    opted_in = False
    if session['role'] == 'student':
        opted_in = database.is_student_opted_in(session['user_id'], id)

    return render_template(
        'curriculum.html',
        curriculum=curr,
        data=parsed_json,
        sum_courses=sum_courses,
        total_credits=total_credits,
        opted_in=opted_in
    )

```

```

@app.route('/roadmap/<int:id>')
@login_required
def roadmap_page(id):
    curr = database.get_curriculum(id)
    if not curr:
        flash("Curriculum not found.", "danger")
        return redirect(url_for('dashboard_page'))

    parsed_json = json.loads(curr['raw_json'])
    return render_template('roadmap.html', curriculum=curr, data=parsed_json)

@app.route('/assistant')
@login_required
def assistant_page():
    # Students see curricula they opted into. Educators see what they generated.
    user_id = session['user_id']
    if session['role'] == 'student':
        # List all educator-generated curricula in the system so students can choose courses
        history = database.get_all_curricula()
    else:
        history = database.get_user_curriculum_history(user_id)

    saved_plans = database.get_learning_plans_by_user(user_id)
    return render_template('assistant.html', curricula=history, saved_plans=saved_plans)

@app.route('/assistant/plan/<int:id>')
@login_required
def assistant_plan_page(id):
    plan = database.get_learning_plan(id)
    if not plan or (plan['user_id'] != session['user_id'] and session['role'] != 'admin'):
        flash("Learning plan not found or access restricted.", "danger")
        return redirect(url_for('assistant_page'))

    parsed_json = json.loads(plan['plan_json'])

    # Get completed checkboxes
    completed_tuples = database.get_completed_tasks(session['user_id'], id)

```



```
def assistant_plan_page(id):
    # Convert list of tuples (week, idx) to structure matching template checks
    completed_tasks = {}
    for week_num, task_idx in completed_tuples:
        if week_num not in completed_tasks:
            completed_tasks[week_num] = []
        completed_tasks[week_num].append(task_idx)

    return render_template(
        'assistant_plan.html',
        plan=plan,
        data=parsed_json,
        completed_tasks=completed_tasks
    )

@app.route('/history')
@login_required
def history_page():
    user_id = session['user_id']
    if session['role'] == 'admin':
        history = database.get_all_curricula()
        opted_ids = set()
    elif session['role'] == 'student':
        history = database.get_all_curricula() # show all, with opt-in status
        opted = database.get_student_opted_curricula(user_id)
        opted_ids = {c['id'] for c in opted}
    else:
        history = database.get_user_curriculum_history(user_id)
        opted_ids = set()

    download_history = database.get_download_history_by_user(user_id)
    saved_plans = database.get_learning_plans_by_user(user_id) if session['role'] == 'student' else []
    return render_template('history.html', history=history, downloads=download_history,
                           saved_plans=saved_plans, opted_ids=opted_ids)
```

```
@app.route('/profile')
@login_required
def profile_page():
    user = database.get_user_by_id(session['user_id'])
    return render_template('profile.html', user=user)

@app.route('/faq')
def faq_page():
    return render_template('faq.html')

@app.route('/feedback')
@login_required
def feedback_page():
    return render_template('feedback.html')

@app.route('/contact')
def contact_page():
    return render_template('contact.html')

@app.route('/subscription')
@login_required
def subscription_page():
    user = database.get_user_by_id(session['user_id'])
    return render_template('subscription.html', user_details=user)

@app.route('/tracker')
@login_required
@educator_or_admin_required
def tracker_page():
    # Fetch curricula created by this educator or all if admin
    user_id = session['user_id']
    if session['role'] == 'admin':
        curricula_list = database.get_all_curricula()
    else:
        curricula_list = database.get_user_curriculum_history(user_id)
```

```
def tracker_page():
    tracker_data = []
    for curr in curricula_list:
        opted_students = database.get_opted_students_by_curriculum(curr['id'])
        quiz_scores = database.get_quiz_scores_by_curriculum(curr['id'])
        tracker_data.append({
            "curriculum": curr,
            "students": opted_students,
            "quizzes": quiz_scores
        })

    return render_template('tracker.html', tracker_data=tracker_data)

# --- API endpoints ---

@app.route('/api/auth/login', methods=['POST'])
def api_login():
    data = request.json or {}
    email = data.get('email')
    password = data.get('password')

    if not email or not password:
        return jsonify({"success": False, "message": "Email and password are required."}), 400

    user = database.authenticate_user(email, password)
    if user:
        session['user_id'] = user['id']
        session['email'] = user['email']
        session['full_name'] = user['full_name']
        session['role'] = user['role']
        return jsonify({"success": True, "message": "Logged in successfully."})

    return jsonify({"success": False, "message": "Invalid email or password."}), 401

@app.route('/api/auth/register', methods=['POST'])
def api_register():
    data = request.json or {}
    email = data.get('email')
```

```
        field_of_study=field,
        duration_semesters=duration_int,
        raw_json=json.dumps(curriculum_json)
    )

    # Increment generation count
    database.increment_generation_count(session['user_id'])

    return jsonify({
        "success": True,
        "curriculum_id": curr_id,
        "message": "Curriculum generated and logged successfully."
    })

@app.route('/api/curriculum/<int:id>')
@login_required
def api_curriculum_detail(id):
    curr = database.get_curriculum(id)
    if not curr:
        return jsonify({"success": False, "message": "Curriculum not found."}), 404
    return jsonify(json.loads(curr['raw_json']))

@app.route('/api/curriculum/<int:id>/courses')
@login_required
def api_curriculum_courses(id):
    curr = database.get_curriculum(id)
    if not curr:
        return jsonify([]), 404
    parsed_json = json.loads(curr['raw_json'])
    courses = []
    for sem in parsed_json.get('semesters', []):
        for course in sem.get('courses', []):
            courses.append(course.get('name'))
    return jsonify(courses)

@app.route('/api/curriculum/optin', methods=['POST'])
@login_required
```



```

@app.route('/api/curriculum/optin', methods=['POST'])
@login_required
def api_curriculum_optin():
    data = request.json or {}
    curriculum_id = data.get('curriculum_id')
    if not curriculum_id:
        return jsonify({"success": False, "message": "Curriculum ID is required."}), 400

    success = database.opt_in_curriculum(session['user_id'], curriculum_id)
    if success:
        return jsonify({"success": True, "message": "Opted-in to curriculum successfully."})
    return jsonify({"success": False, "message": "Already opted-in to this curriculum."})

@app.route('/api/curriculum/optout', methods=['POST'])
@login_required
def api_curriculum_optout():
    data = request.json or {}
    curriculum_id = data.get('curriculum_id')
    if not curriculum_id:
        return jsonify({"success": False, "message": "Curriculum ID is required."}), 400

    database.opt_out_curriculum(session['user_id'], curriculum_id)
    return jsonify({"success": True, "message": "Opted-out of curriculum successfully."})

@app.route('/api/curriculum/<int:id>/rate', methods=['POST'])
@login_required
def api_curriculum_rate(id):
    data = request.json or {}
    rating = data.get('rating')
    if not rating:
        return jsonify({"success": False, "message": "Rating value required."}), 400
    try:
        rating_int = int(rating)
        database.rate_curriculum(id, rating_int)
        return jsonify({"success": True, "message": "Rating updated successfully."})
    except ValueError:
        return jsonify({"success": False, "message": "Invalid rating score."}), 400

    question = data.get('question')

    if not course_name or not question:
        return jsonify({"success": False, "message": "Course name and question are required."})

    answer = llm_service.generate_custom_interview_answer(course_name, question)
    return jsonify({"success": True, "answer": answer})

# --- Quiz APIs ---

@app.route('/api/quiz/get')
@login_required
def api_quiz_get():
    course_name = request.args.get('course_name')
    course_names = request.args.get('course_names') # comma-separated for semester quiz
    week_topic = request.args.get('week_topic', course_name)
    if not course_name and not course_names:
        return jsonify([]), 400
    if course_names:
        names_list = [n.strip() for n in course_names.split(',') if n.strip()]
        quiz = llm_service.generate_semester_quiz(names_list)
    else:
        quiz = llm_service.generate_course_quiz(week_topic or course_name)
    return jsonify(quiz)

@app.route('/api/quiz/submit', methods=['POST'])
@login_required
def api_quiz_submit():
    data = request.json or {}
    curriculum_id = data.get('curriculum_id')
    course_code = data.get('course_code')
    score = data.get('score')
    total_questions = data.get('total_questions')
    explanation = data.get('wrong_explanations', '')

    if not curriculum_id or not course_code or score is None or total_questions is None:
        return jsonify({"success": False, "message": "Missing arguments."}), 400

```

```

    return send_file(
        io.BytesIO(output.getvalue().encode('utf-8')),
        mimetype='text/csv',
        as_attachment=True,
        download_name=filename
    )

# --- AI-Powered Standalone Report Generator ---

@app.route('/api/report/generate', methods=['POST'])
@login_required
def api_report_generate():
    """
    Fully AI-driven report generator.
    Accepts a free-text topic + format, builds everything from scratch.
    """
    data = request.json or {}
    topic = (data.get('topic') or '').strip()
    fmt = (data.get('format') or 'pdf').lower()
    curr_id = data.get('curriculum_id') # optional - used to seed curriculum data

    if not topic:
        return jsonify({"success": False, "message": "Topic is required."}), 400

    # Build curriculum base: use existing curriculum if provided, else generate from topic
    if curr_id:
        curr = database.get_curriculum(int(curr_id))
        curriculum_data = json.loads(curr['raw_json']) if curr else None
    else:
        curriculum_data = None

    if not curriculum_data:
        curriculum_data = llm_service.generate_curriculum_from_llm(
            title=topic, field=topic, duration=4
        )

    # Generate enriched report sections from the topic

@app.route('/api/admin/user/delete', methods=['POST'])
@login_required
def api_admin_delete_user():
    if session['role'] != 'admin':
        return jsonify({'success': False, 'message': 'Unauthorized'}), 403
    data = request.json or {}
    user_id = data.get('user_id')
    if not user_id or int(user_id) == session['user_id']:
        return jsonify({'success': False, 'message': 'Cannot delete yourself.'}), 400
    conn = database.get_db_connection()
    conn.execute('DELETE FROM users WHERE id = ?', (user_id,))
    conn.commit()
    conn.close()
    return jsonify({'success': True, 'message': 'User deleted successfully'})

if __name__ == '__main__':
    debug_mode = os.environ.get("FLASK_DEBUG", "0") == "1"
    port = int(os.environ.get("PORT", 5000))
    app.run(debug=debug_mode, host="0.0.0.0", port=port)

```

Milestone 3: Frontend Development

This milestone develops a responsive web interface using HTML, CSS, JavaScript and Bootstrap.

Activity 3.1: User Interface Development

```


<div class="glass-card" id="auth-card">

    <!-- Toggle tabs -->
    <div style="display:flex; justify-content:space-around; margin-bottom:2rem; border-bottom:2px solid var(--glass-border); padding-bottom:1rem;">
      <button class="btn" id="tab-login-btn" style="background:none; border:none; color:var(--text-primary); font-size:1.1rem; border-bottom:2px solid var(--color-primary); border-radius:8; padding:0.5rem 1rem; onclick="switchAuthTab('login')">
        <i class="fa-solid fa-lock"/> Sign In
      </button>
      <button class="btn" id="tab-register-btn" style="background:none; border:none; color:var(--text-muted); font-size:1.1rem; border-bottom:2px solid transparent; border-radius:8; padding:0.5rem 1rem; onclick="switchAuthTab('register')">
        <i class="fa-solid fa-user-plus"/> Register
      </button>
    </div>

    <div id="auth-error-alert" class="alert alert-danger" style="display:none; margin-bottom:1.5rem;">
      <span id="auth-error-msg">Error message goes here.</span>
    </div>

    <!-- Login Form -->
    <form id="login-form" onsubmit="handleAuthSubmit(event, 'login')">
      <h2 class="form-title" style="margin-bottom:1.5rem; text-align:center;">Welcome to LERNIX</h2>
      <div class="form-group">
        <label class="form-label" for="login-email">Email Address</label>
        <input type="email" id="login-email" class="form-input" placeholder="e.g. educator@lernix.edu" required>
      </div>

      <div class="form-group">
        <label class="form-label" for="login-password">Password</label>
        <input type="password" id="login-password" class="form-input" placeholder="*****" required>
      </div>

      <button type="submit" class="btn btn-primary" style="width:100%; margin-top:1rem;" id="login-submit-btn">
        <i class="fa-solid fa-right-to-bracket"/> Sign In
      </button>

      <div class="auth-links">
        <a href="{url_for('forgot_password_page')}" id="login-forgot-pwd">Forgot Password</a>
        <span style="color:var(--text-muted);">Demo credentials: <i>student@lernix.edu</i> (Pw: <i>student123</i>) | <i>educator@lernix.edu</i> (Pw: <i>educator123</i>)</span>
      </div>
    </form>

    <!-- Registration Form (Hidden by default) -->
  </div>


```



```
<!-- Registration Form (Hidden by default) -->
<form id="register-form" onsubmit="handleAuthSubmit(event, 'register')" style="display:none;"
  <h2 class="form-title" style="margin-bottom:1.5rem; text-align:center;">Create Account</h2>
  <div class="form-group">
    <label class="form-label" for="reg-name">Full Name</label>
    <input type="text" id="reg-name" class="form-input" placeholder="e.g. Dr. John Doe" required>
  </div>

  <div class="form-group">
    <label class="form-label" for="reg-email">Email Address</label>
    <input type="email" id="reg-email" class="form-input" placeholder="e.g. student@lernix.edu" required>
  </div>

  <div class="form-group">
    <label class="form-label" for="reg-password">Password</label>
    <input type="password" id="reg-password" class="form-input" placeholder="Min 6 characters" required minlength="6">
  </div>

  <div class="form-group">
    <label class="form-label" for="reg-role">Account Role</label>
    <select id="reg-role" class="form-input" onchange="toggleStudentFields(this.value)">
      <option value="student">Student (Select roadmaps & tracks)</option>
      <option value="educator">Educator (Generate curricula & track progress)</option>
    </select>
  </div>
</div>
```

```
function switchAuthTab(tab) {
  const loginForm = document.getElementById('login-form');
  const registerForm = document.getElementById('register-form');
  const loginTab = document.getElementById('tab-login-btn');
  const registerTab = document.getElementById('tab-register-btn');
  const alertDiv = document.getElementById('auth-error-alert');

  alertDiv.style.display = 'none';

  if (tab === 'login') {
    loginForm.style.display = 'block';
    registerForm.style.display = 'none';
    loginTab.style.borderColorBottomColor = 'var(--color-primary)';
    loginTab.style.color = 'var(--text-primary)';
    registerTab.style.borderColorBottomColor = 'transparent';
    registerTab.style.color = 'var(--text-muted)';
  } else {
    loginForm.style.display = 'none';
    registerForm.style.display = 'block';
    loginTab.style.borderColorBottomColor = 'transparent';
    loginTab.style.color = 'var(--text-muted)';
    registerTab.style.borderColorBottomColor = 'var(--color-accent)';
    registerTab.style.color = 'var(--text-primary)';
    toggleStudentFields(document.getElementById('reg-role').value);
  }
}
```

```
{% block extra_js %}
<script src="{{ url_for('static', filename='js/dashboard.js') }}"></script>
{% if session.get('role') == 'student' %}
<script>
function showMistakeReport(courseCode, quizId) {
    const dataContainer = document.getElementById('mistake-data-' + quizId);
    if (!dataContainer) return;

    document.getElementById('mistake-modal-title').innerHTML = '<i class="fa-solid fa-circle-exclamation"></i> Mistake Analysis: ${courseCode}';
    document.getElementById('mistake-modal-body').textContent = dataContainer.textContent;
    document.getElementById('mistake-modal').style.display = 'flex';
}

function closeMistakeModal() {
    document.getElementById('mistake-modal').style.display = 'none';
}

// Close modal if clicking outside container
window.addEventListener('click', function(e) {
    const modal = document.getElementById('mistake-modal');
    if (e.target === modal) {
        closeMistakeModal();
    }
});
</script>
{% endif %}
{% endblock %}
```

Milestone 4: Application Deployment & Execution

This milestone focuses on running the complete Lernix system locally and verifying all modules.

Start FastAPI Backend

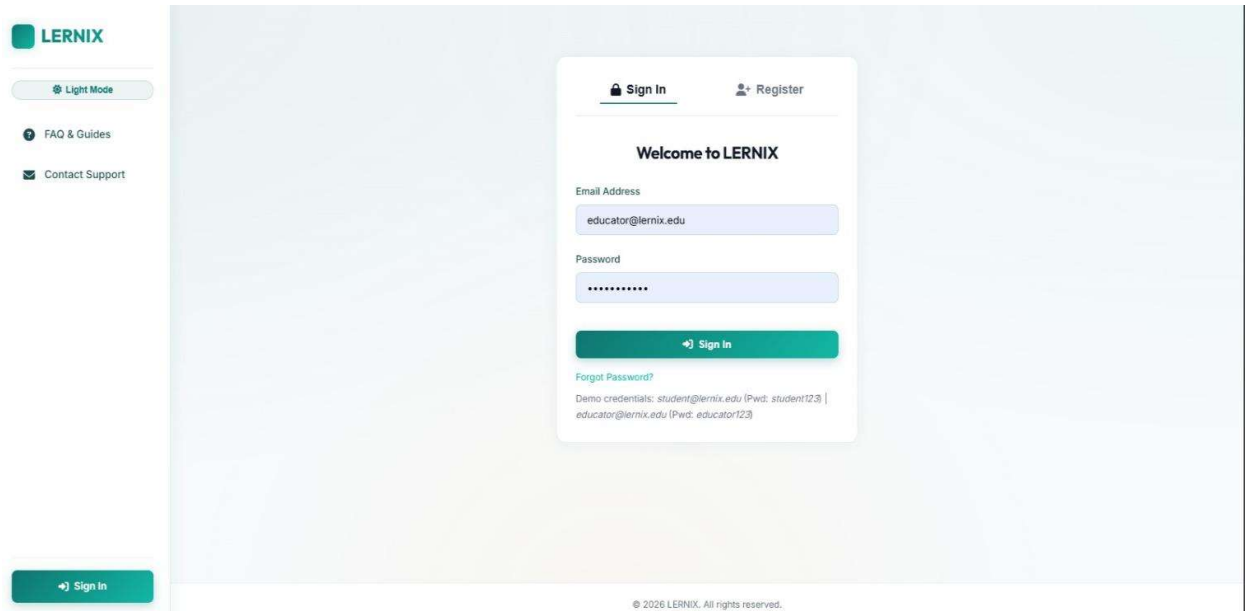
```
PS C:\Users\Dell\gemini\antigravity\scratch\lernix> python app.py
>>
Database migrated and validated successfully.
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.1.38.244:5000
Press CTRL+C to quit
* Restarting with watchdog (windowsapi)
Database migrated and validated successfully.
* Debugger is active!
* Debugger PIN: 869-949-595
█
```

Access the Web Application

<http://127.0.0.1:5000>

Project Output

Login Page



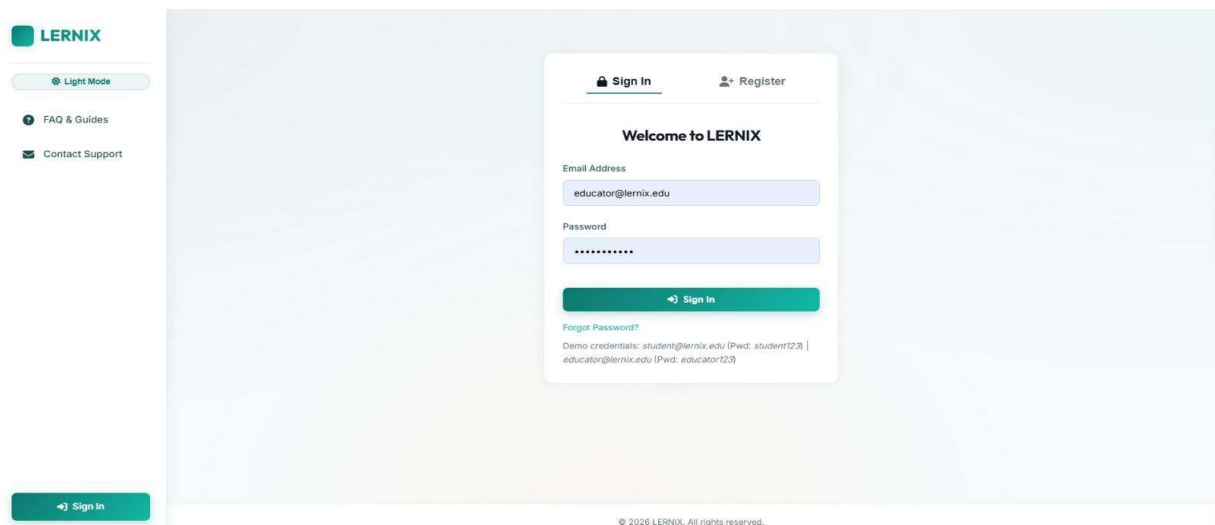
The screenshot displays the LERNIX login interface. On the left, a sidebar contains the LERNIX logo, a 'Light Mode' toggle, and links for 'FAQ & Guides' and 'Contact Support'. The main content area features a central login card with a 'Sign In' button and a 'Register' link. The card includes fields for 'Email Address' (containing 'educator@lernix.edu') and 'Password' (masked with dots). Below the fields is a 'Sign In' button. A 'Forgot Password?' link is provided, along with demo credentials: 'student@lernix.edu (Pwd: student123)' and 'educator@lernix.edu (Pwd: educator123)'. A 'Sign In' button is also located at the bottom left of the page. The footer indicates '© 2026 LERNIX. All rights reserved.'

The Educator Login page provides a secure login interface with email and password fields. JWT authentication is initiated upon successful credential validation.

The Student Login page mirrors the educator login interface with appropriate role-based routing upon authentication.

Registration Page

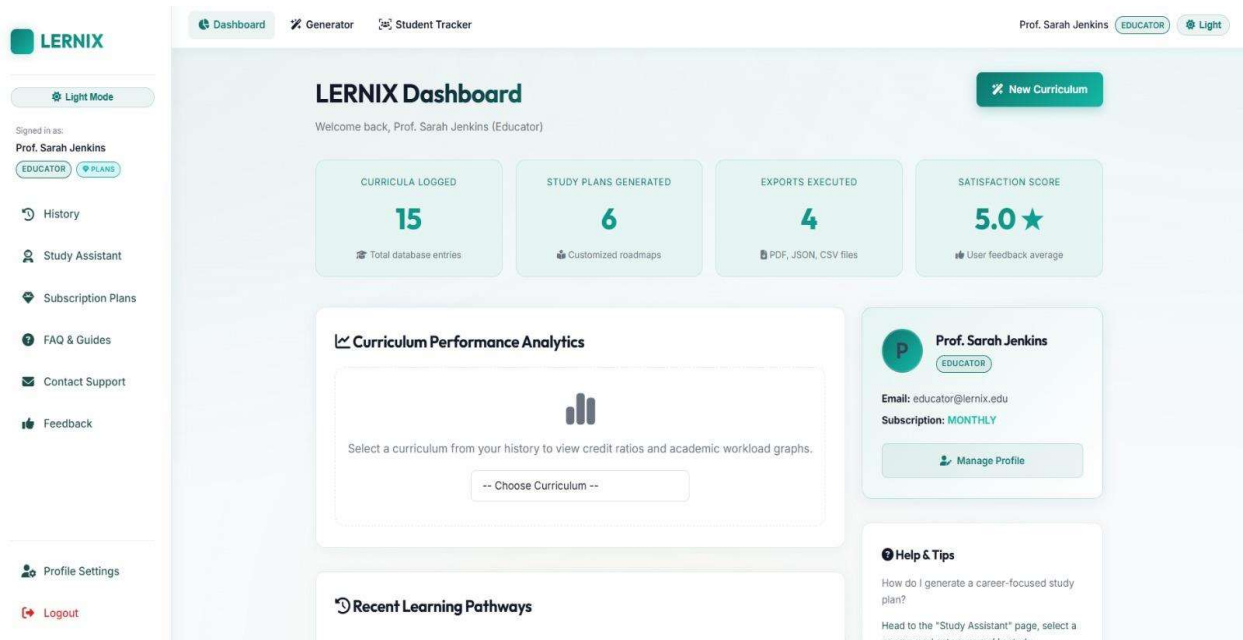
The Registration Page allows new users to register with name, email, password, and role Selection (Educator or Student).



This screenshot is identical to the one above, showing the LERNIX login interface. It includes the sidebar with the LERNIX logo, 'Light Mode' toggle, and links for 'FAQ & Guides' and 'Contact Support'. The central login card features a 'Sign In' button and a 'Register' link. The 'Email Address' field contains 'educator@lernix.edu' and the 'Password' field is masked. A 'Sign In' button is present below the fields. A 'Forgot Password?' link and demo credentials ('student@lernix.edu (Pwd: student123)' and 'educator@lernix.edu (Pwd: educator123)') are also visible. A 'Sign In' button is located at the bottom left of the page. The footer states '© 2026 LERNIX. All rights reserved.'

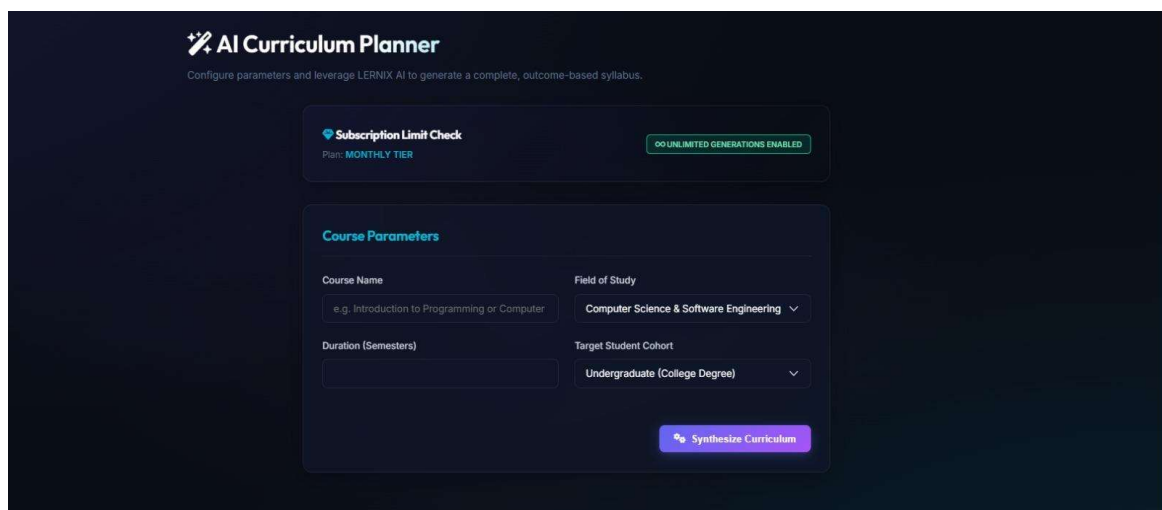
Educator Dashboard

The Educator Dashboard displays the user's generated curricula history, quick access to the curriculum generation module, analytics summary cards, and navigation to all educator features.



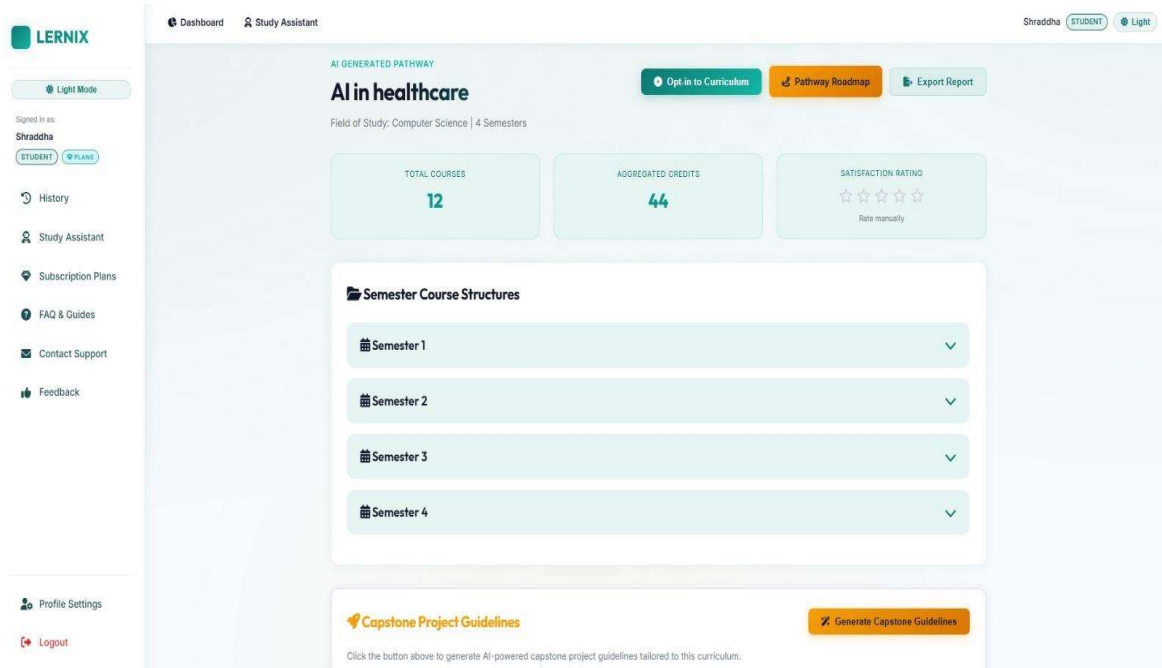
Curriculum Generation Page

The Curriculum Generation Page presents the course input form where educators enter Course Name, Educational Level, Number of Semesters, Weekly Hours, Total Credits, and optional Technology Specialization. A Generate Curriculum button triggers the AI processing.



Generated Curriculum Output

The Generated Curriculum Output displays the complete AI-generated curriculum in a structured, semester-wise accordion layout. Each semester panel expands to show course names, modules, topics.



LERNIX | Dashboard | Study Assistant | Shradha | STUDENT | Light

AI GENERATED PATHWAY
AI in healthcare
Field of Study: Computer Science | 4 Semesters

Opt in to Curriculum | Pathway Roadmap | Export Report

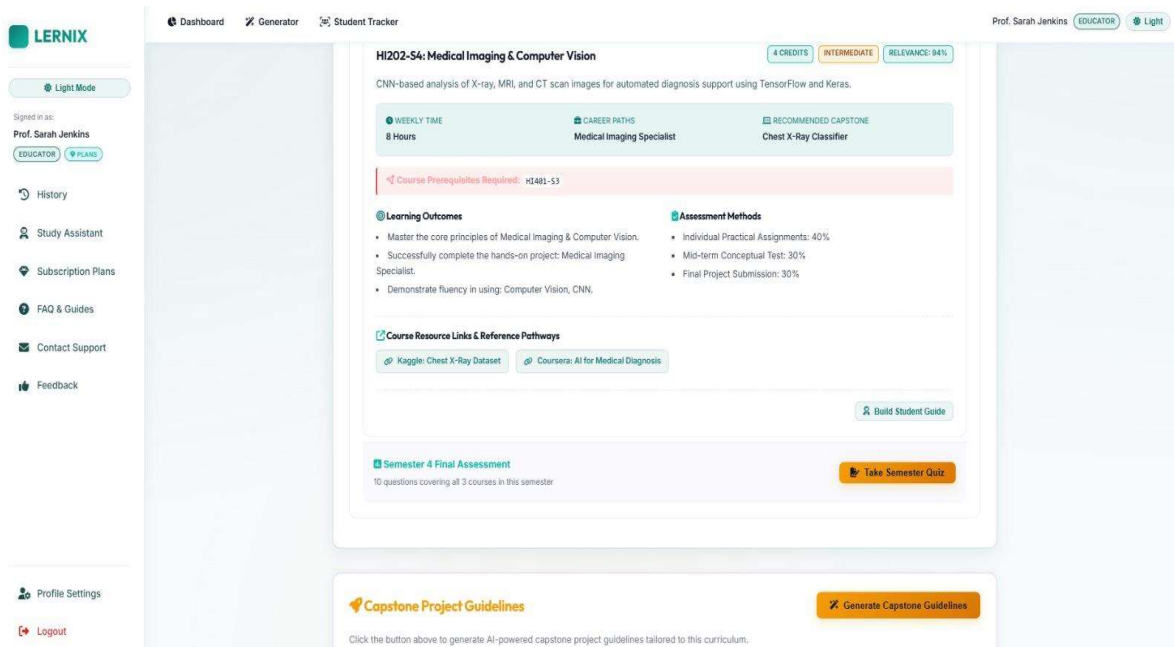
TOTAL COURSES: **12** | AGGREGATED CREDITS: **44** | SATISFACTION RATING: ☆☆☆☆☆ (Rate manually)

Semester Course Structures

- Semester 1
- Semester 2
- Semester 3
- Semester 4

Capstone Project Guidelines | Generate Capstone Guidelines

Click the button above to generate AI-powered capstone project guidelines tailored to this curriculum.



LERNIX | Dashboard | Generator | Student Tracker | Prof. Sarah Jenkins | EDUCATOR | Light

HI202-54: Medical Imaging & Computer Vision | 4 CREDITS | INTERMEDIATE | RELEVANCE: 94%

CNN-based analysis of X-ray, MRI, and CT scan images for automated diagnosis support using TensorFlow and Keras.

WEEKLY TIME: 8 Hours | CAREER PATHS: Medical Imaging Specialist | RECOMMENDED CAPSTONE: Chest X-Ray Classifier

Course Prerequisites Required: HI401-53

Learning Outcomes

- Master the core principles of Medical imaging & Computer Vision.
- Successfully complete the hands-on project: Medical imaging Specialist.
- Demonstrate fluency in using: Computer Vision, CNN.

Assessment Methods

- Individual Practical Assignments: 40%
- Mid-term Conceptual Test: 30%
- Final Project Submission: 30%

Course Resource Links & Reference Pathways

- Kaggle: Chest X-Ray Dataset
- Coursera: AI for Medical Diagnosis

Build Student Guide

Semester 4 Final Assessment
10 questions covering all 3 courses in this semester | Take Semester Quiz

Capstone Project Guidelines | Generate Capstone Guidelines

Click the button above to generate AI-powered capstone project guidelines tailored to this curriculum.



Light Mode

Signed in as:

Prof. Sarah Jenkins

EDUCATOR

FLAME

History

Study Assistant

Subscription Plans

FAQ & Guides

Contact Support

Feedback

Profile Settings

Logout

Dashboard

Generator

Student Tracker

Prof. Sarah Jenkins

EDUCATOR

Light

Curriculum Logs & Exports

Inspect previously formulated educational plans, templates, and download histories.

Generated Curricula

Export History

Generated Curricula

Course Name	Field	Date	Actions
Introduction to robotics	Me	2026-06-11	View Roadmap Delete
AI in healthcare	Computer Science	2026-06-11	View Roadmap Delete
Physics	Physics	2026-06-10	View Roadmap Delete
Disaster management	Disaster management	2026-06-10	View Roadmap Delete
vibe coding	Artificial Intelligence	2026-06-10	View Roadmap Delete
visi	ECE	2026-06-10	View Roadmap Delete
programming for problem solving through c	Computer Science	2026-06-10	View Roadmap Delete
AI in Healthcare	Artificial Intelligence	2026-06-10	View Roadmap Delete
nlp	Computer Science	2026-06-10	View Roadmap Delete

Student Tracker

Dashboard

Generator

Student Tracker

Prof. Sarah Jenkins

EDUCATOR

Light

AI in Healthcare

ID: #18 | Field: Artificial Intelligence

Enrolled Student Progress

No students have opted into this curriculum yet.

Course Quiz Scores

No quiz attempts recorded for this curriculum yet.

nlp

ID: #17 | Field: Computer Science

Enrolled Student Progress

Student Name	Roll Number	Branch	Academic Year	Tasks Completed	Progress Bar
Shraddha abzaheda@gmail.com	582	EEE	3rd Year	0 / 0 tasks	0%

Course Quiz Scores

No quiz attempts recorded for this curriculum yet.

Student Weekly Learning Plan

The Student Weekly Plan page displays the AI-generated personalized study schedule with weekly objectives, topics, activities, resources, and mini-assessments presented in a clear week-by-week timeline.



Light Mode

Signed in as: Shraddha

STUDENT PLANS

History

Study Assistant

Subscription Plans

FAQ & Guides

Contact Support

Feedback

Profile Settings

Logout

Dashboard Study Assistant

Shraddha STUDENT Light

Curriculum Logs & Exports

Review the curricula you have opted into, your saved study plans, and export histories.

Available Curricula Saved Study Plans Export History

Available Curricula

Course Name	Field	Date	Actions
Introduction to robotics	Me	2026-06-11	View Roadmap Study Plan Opted In
AI in healthcare	Computer Science	2026-06-11	View Roadmap Study Plan Opted In
Physics	Physics	2026-06-10	View Roadmap Study Plan + Opt In
Disaster management	Disaster management	2026-06-10	View Roadmap Study Plan + Opt In
vibe coding	Artificial Intelligence	2026-06-10	View Roadmap Study Plan + Opt In
vlsi	ECE	2026-06-10	View Roadmap Study Plan + Opt In
programming for problem solving through c	Computer Science	2026-06-10	View Roadmap Study Plan + Opt In
AI in Healthcare	Artificial Intelligence	2026-06-10	View Roadmap Study Plan + Opt In
nlp	Computer Science	2026-06-10	View Roadmap Study Plan Opted In



Light Mode

Signed in as: Shraddha

STUDENT PLANS

History

Study Assistant

Subscription Plans

FAQ & Guides

Contact Support

Feedback

Profile Settings

Logout

Dashboard Study Assistant

Shraddha STUDENT Light

Study Plan: AI in healthcare

Personalized Study Plan | Weekly Allocation: 10 Hours

Weekly Study Tasks & Progress

Week 1 Milestone Target

Health Data Foundations & EHR Systems

- ☐ Overview of EHR systems and HL7/FHIR standards (3 hrs)
- ☐ Loading and exploring clinical datasets with Pandas (3 hrs)
- ☐ Data cleaning: missing values, outliers in patient records (4 hrs)

TOPIC REFERENCE RESOURCES:

Google Scholar: AI in healthcare Courseera Courses

Target Milestone: Perform exploratory analysis on a public health dataset (e.g. MIMIC-III sample)

WEEK 1 SELF-ASSESSMENT QUIZ Start Quiz

Week 2 Milestone Target

AI & ML Models for Disease Prediction

- ☐ Supervised learning for diagnosis classification (diabetes, heart disease) (3 hrs)

Revision Checkpoints

- Phase 1: Foundation Building (Week 1)**
 Focus: Core terminology, tooling, and environment setup for AI in healthcare.
 COMPLETE SETUP CHECKLIST FINISH WEEK 1 QUIZ
- Phase 2: Concept Deep Dive (Weeks 2-3)**
 Focus: Architecture, design patterns, and applied coding in AI in healthcare.
 SUBMIT WEEK 2 MINI-PROJECT PASS MID-POINT ASSESSMENT
- Phase 3: Project & Review (Week 4)**
 Focus: Integration, optimisation, and capstone demonstration for AI in healthcare.
 RUN FULL TEST SUITE PRESENT FINAL PROJECT DEMO

Interview Preparation Q&A

Have a specific question? Ask LERNIX AI:

e.g. What is the difference between tuple and list? Ask

Student: how to make biryani

LERNIX AI: Sorry, I can't answer that. Please ask a question related to your course or academic topic.

Topic: Core AI in healthcare Concepts

Assessment Page

The Assessment Page presents AI-generated quizzes, assignments, and exam questions for the selected course. Students can attempt assessments and view feedback.

Semester 1 Assessment Quiz

1. What does EHR stand for in healthcare IT?

- ☐ Electronic Health Record
- ☐ Encoded Health Report
- ☐ External Health Registry
- ☐ Encrypted Hospital Record

2. What is data de-identification in healthcare?

- ☐ Deleting all patient records
- ☐ Removing PII so data can be used for research without exposing identity
- ☐ Encrypting data for transfer
- ☐ Converting data to JSON

3. What is FHIR primarily used for?

- ☐ Encrypting patient files
- ☐ Standardising healthcare data exchange between systems
- ☐ Training AI models
- ☐ Hospital billing only

4. Why is model explainability important in healthcare AI?

- ☐ It speeds up training

CapStone Project Guidelines

This presents a structured overview of a professional capstone project, including its goals, required deliverables, recommended tools, and development phases. It serves as a roadmap for students to plan, build, document, and present a complete real-world project in a systematic manner.

🔥 Capstone Project Guidelines

[🔄 Regenerate](#)

Capstone Project: AI in healthcare

This capstone project challenges students to integrate all concepts learned across the AI in healthcare curriculum into a single, deployable, industry-grade solution in the domain of Computer Science.

🎯 Objectives

- Apply theoretical knowledge to a real-world problem
- Demonstrate full-stack or end-to-end system design skills
- Produce professional documentation and a working prototype
- Present findings to a panel of evaluators

🔧 Recommended Tools

GIT / GITHUB VS CODE DOCKER POSTMAN FIGMA
JUPYTER NOTEBOOK

💡 Suggested Topics

- AI-powered Computer Science recommendation system
- Full-stack Computer Science management platform
- Data analytics dashboard for Computer Science metrics

📦 Deliverables

Project Proposal: 1-2 page document outlining problem statement, objectives, and tech stack

Working Prototype: Functional application or model meeting defined requirements

Technical Report: 10-15 page report covering architecture, implementation, and evaluation

Final Presentation: 15-minute demo and Q&A session with evaluators

📅 Milestones

Research & Planning (2 weeks)

- Define problem scope
- Review literature
- Select technology stack

Design & Architecture (2 weeks)

- Create system diagrams
- Define data models
- Set up project repository

PDF Export Output

This document provides a detailed overview of a professional learning and development program, including curriculum structure, learning objectives, skill development pathways, resources, and project recommendations. It serves as a comprehensive guide for building domain expertise and achieving career readiness through a structured educational framework.

LERNIX — Professional Curriculum Report

June 11, 2026

1. Program Overview

Program Title	AI in healthcare
Field of Study	Computer Science
Duration	4 Semesters
Total Courses	12
Total Credits	44
Target Audience	Undergraduate & Professional
Design Paradigm	Outcome-Based Education (OBE)
Standard	ACM / IEEE Joint Curriculum Guidelines

Program Description

This comprehensive AI in healthcare program provides students with both theoretical foundations and industry-ready practical skills in Computer Science. Students progress through structured semesters covering core principles, applied development, and advanced system design. Graduates are prepared for immediate employment or advanced academic research.

Learning Objectives

- Master foundational and advanced concepts of AI in healthcare
- Apply industry-standard tools and frameworks in Computer Science
- Design and implement scalable, production-grade systems
- Collaborate effectively in Agile / DevOps environments
- Demonstrate research and innovation capabilities

2. Semester-wise Curriculum

Semester 1

Code	Course Title	Credits	Difficulty	Weekly Hrs	Relevance
HI101-S1	Introduction to Health Informatics	3	Beginner	6 hrs	91%
HI102-S1	Medical Data Analysis with Python	4	Beginner	8 hrs	95%
HI201-S1	AI & Machine Learning in Healthcare	4	Intermediate	8 hrs	93%

HI101-S1: Introduction to Health Informatics

Credits	3
Difficulty	Beginner
Weekly Hours	6 hrs
Relevance	91%
Prerequisites	None

Detailed Description

Overview of healthcare systems, electronic health records (EHR), health data standards (HL7, FHIR), and patient data management.

Learning Outcomes

- Master the core principles of Introduction to Health Informatics.
- Successfully complete the hands-on project: Health Informatics Analyst.
- Demonstrate fluency in using: Health Data, HL7.

Career Pathways: Health Informatics Analyst

Recommended Capstone: EHR System Prototype

Assessment Structure: Individual Practical Assignments: 40%, Mid-term Conceptual Test: 30%, Final Project Submission: 30%

HI102-S1: Medical Data Analysis with Python

Credits 4

Difficulty Beginner

Weekly Hours 8 hrs

Relevance 95%

Prerequisites None

Detailed Description

Applying Python (Pandas, NumPy) to clean, analyse, and visualise clinical datasets including patient records and lab results.

Learning Outcomes

- Master the core principles of Medical Data Analysis with Python.
- Successfully complete the hands-on project: Clinical Data Analyst.
- Demonstrate fluency in using: Python, Medical Data.

Career Pathways: Clinical Data Analyst

Recommended Capstone: Patient Readmission Analysis

Assessment Structure: Individual Practical Assignments: 40%, Mid-term Conceptual Test: 30%, Final Project Submission: 30%

HI201-S1: AI & Machine Learning in Healthcare

Credits 4

Difficulty Intermediate

Weekly Hours 8 hrs

Relevance 93%

Prerequisites None

Detailed Description

Supervised and unsupervised ML applied to disease prediction, medical image classification, and patient outcome modelling.

Learning Outcomes

- Master the core principles of AI & Machine Learning in Healthcare.
- Successfully complete the hands-on project: Healthcare AI Engineer.
- Demonstrate fluency in using: Machine Learning, Scikit-Learn.

Career Pathways: Healthcare AI Engineer

Recommended Capstone: Diabetes Risk Predictor

Assessment Structure: Individual Practical Assignments: 40%, Mid-term Conceptual Test: 30%, Final Project Submission: 30%

Course-Level Reference Links:

Provides relevant learning resources, documentation, datasets, and online courses to help learners gain deeper knowledge and practical skills in each subject area.

Course-level Reference Links

Course Code	Resource Title	URL
HI101-S1	HL7 FHIR Documentation	https://hl7.org/fhir/
HI101-S1	Health IT on Coursera	https://www.coursera.org/search?query=health+informatics
HI102-S1	Kaggle Healthcare Datasets	https://www.kaggle.com/search?q=healthcare
HI102-S1	Google Scholar: Medical Data Analysis	https://scholar.google.com/scholar?q=medical+data+analysis+python
HI201-S1	Coursera: AI for Medicine	https://www.coursera.org/specializations/ai-for-medicine
HI201-S1	Google Scholar: ML Healthcare	https://scholar.google.com/scholar?q=machine+learning+healthcare
HI202-S2	Kaggle: Chest X-Ray Dataset	https://www.kaggle.com/datasets/paulthothymoney/chest-xray-pneumonia
HI202-S2	Coursera: AI for Medical Diagnosis	https://www.coursera.org/learn/ai-for-medical-diagnosis
HI301-S2	HuggingFace BioBERT	https://huggingface.co/dmis-lab/biobert-v1.1
HI301-S2	Google Scholar: Clinical NLP	https://scholar.google.com/scholar?q=clinical+NLP+BERT
HI302-S2	HHS HIPAA Guidelines	https://www.hhs.gov/hipaa/index.html
HI302-S2	WHO Ethics & AI in Health	https://www.who.int/publications/l/item/9789240029200
HI401-S3	HL7 CDS Hooks	https://cds-hooks.hl7.org/
HI401-S3	Google Scholar: CDSS	https://scholar.google.com/scholar?q=clinical+decision+support+systems+AI
HI402-S3	FDA AI/ML Action Plan	https://www.fda.gov/medical-devices/software-medical-device-samd/artificial-intelligence-and-machine-learning-ai-ml-enabled-medical-devices
HI402-S3	Google Scholar: FDA AI	https://scholar.google.com/scholar?q=FDA+AI+ML+medical+devices
HI101-S3	HL7 FHIR Documentation	https://hl7.org/fhir/
HI101-S3	Health IT on Coursera	https://www.coursera.org/search?query=health+informatics

Career Roadmap

This section outlines the professional growth path within the field, showcasing roles from entry-level positions to leadership opportunities. It highlights key responsibilities and salary progression, providing learners with a clear understanding of potential career advancement and industry opportunities after completing the program.

5. Career Roadmap

Beginner Roles

Role Title	Description	Avg. Salary
AI in healthcare Junior Developer	Entry-level implementation and support roles.	\$55k–\$75k/yr
Technical Analyst	Data gathering, report generation.	\$50k–\$70k/yr

Intermediate Roles

Role Title	Description	Avg. Salary
AI in healthcare Engineer	Design and develop core systems.	\$80k–\$110k/yr
Solutions Architect	Blueprint scalable enterprise solutions.	\$90k–\$120k/yr

Advanced Roles

Role Title	Description	Avg. Salary
Lead AI in healthcare Architect	Drive technical strategy and roadmap.	\$130k–\$160k/yr
Engineering Director	Manage teams, budgets, and delivery.	\$150k+/yr

Salary Insights

Entry Level	\$55k–\$75k / year
Mid Level	\$80k–\$110k / year
Senior / Lead	\$120k–\$160k / year
Director / Principal	\$160k+ / year

Project Summary and Technical Implementation

LERNIX is a Generative AI-powered curriculum and learning management platform developed to automate curriculum creation and provide personalized learning experiences for students. The platform addresses the challenges of manual curriculum design and study planning by leveraging Artificial Intelligence to generate structured curricula, learning roadmaps, quizzes, and study schedules tailored to individual learning goals. By integrating the IBM Granite Large Language Model through Ollama, LERNIX delivers intelligent educational content generation while maintaining data privacy and reducing dependency on external cloud-based AI services.

The platform primarily serves students and administrators. Students can register on the platform, specify their learning goals, preferred topics, skill levels, and study duration, and receive AI-generated curricula, personalized study plans, learning roadmaps, recommended resources, and assessments. The system dynamically creates structured learning pathways that help students achieve their educational objectives efficiently. Students can also track their learning progress, complete AI-generated quizzes, receive performance feedback, and download curriculum reports in PDF format. Administrators manage users, monitor platform activities, oversee generated content, and ensure smooth platform operations through a centralized dashboard.

Technically, LERNIX follows a modern three-tier web architecture. The frontend is developed using HTML5, CSS3, JavaScript, Bootstrap, and Jinja2 templates to provide a responsive and user-friendly interface. The backend is implemented using Python and Flask, which manages routing, business logic, session management, authentication, and communication with the AI model. SQLite is used as the database for storing user accounts, generated curricula, study plans, quiz results, and progress data. The AI functionality is powered by Ollama running the IBM Granite model, which generates curricula, study plans, quizzes, learning resources, and academic assistance through carefully designed prompts. Additionally, PDF generation modules enable users to download professionally formatted curriculum reports. Overall, LERNIX combines Generative AI, web technologies, database management, and educational methodologies to provide a scalable and intelligent learning platform.

Challenges and Optimizations

During the development of LERNIX, several challenges were encountered while integrating Generative AI into curriculum generation and personalized learning workflows. One of the major challenges was generating educational content that was structured, relevant, and suitable for different learning levels. This challenge was addressed through effective prompt engineering techniques and careful formatting of AI-generated responses to ensure consistency in curricula, roadmaps, quizzes, and learning plans.

Another significant challenge was integrating the IBM Granite model through Ollama and managing AI response times efficiently. Since the AI model runs locally, optimization techniques were implemented to streamline prompt processing and improve response generation performance. Maintaining consistency between curriculum modules, learning roadmaps, study schedules, and quizzes also required careful validation of user inputs and generated outputs.

User authentication and access control presented additional challenges. These were resolved through secure session management, password hashing, and role-based access mechanisms. Database performance was optimized through efficient SQLite schema design and CRUD operations, ensuring reliable storage and retrieval of user and curriculum data. The frontend was enhanced using Bootstrap, JavaScript, and responsive design principles to provide an intuitive user experience across different devices. PDF report generation was also optimized to produce well-structured and professional curriculum documents. These improvements collectively enhanced system performance, reliability, security, and usability.

Learning Outcomes

The development of LERNIX provided valuable hands-on experience in building a complete full-stack Generative AI-powered web application. The project strengthened knowledge of Python programming, Flask development, web application architecture, and database management using SQLite. It also provided practical experience in integrating Large Language Models through Ollama and utilizing the IBM Granite model for educational content generation.

The project enhanced understanding of prompt engineering, AI-generated content validation, curriculum design automation, and personalized learning systems. Significant experience was gained in frontend development using HTML, CSS, JavaScript, Bootstrap, and Jinja2 templates to create responsive and interactive user interfaces. The implementation of authentication systems, session management, PDF generation, and role-based access control further improved software engineering and security skills.

Additionally, the project improved problem-solving abilities related to AI integration, system optimization, and educational technology development. Overall, LERNIX strengthened expertise in Generative AI, web development, database systems, educational applications, and modern software engineering practices while demonstrating how AI can be effectively applied to solve real-world educational challenges.

Future Scope

LERNIX can be further enhanced by incorporating advanced AI capabilities and additional educational features to improve learning outcomes. Future enhancements may include support for multiple Large Language Models, multilingual curriculum generation, voice-enabled AI assistants, and adaptive learning systems that adjust study plans based on student performance and learning behavior.

The platform can also be expanded with educator-specific features such as curriculum sharing, collaborative content creation, and advanced student performance monitoring. Future versions may include cloud-based deployment, real-time analytics dashboards, mobile application support, and integration with Learning Management Systems (LMS). Additional capabilities such as AI-powered recommendation systems, automated feedback generation, skill-gap analysis, and predictive learning analytics can further improve personalization and educational effectiveness.

The adoption of scalable cloud infrastructure and enhanced security mechanisms would enable support for larger user communities and institutional deployments. These improvements would transform LERNIX into a comprehensive AI-powered educational ecosystem capable of supporting diverse learning environments and future digital education requirements.

Conclusion

LERNIX successfully demonstrates the potential of Generative AI in transforming curriculum generation, study planning, and personalized learning. By integrating the IBM Granite Large Language Model through Ollama, the platform automates the creation of structured curricula, learning roadmaps, quizzes, study plans, and educational recommendations, significantly reducing the effort required in traditional educational planning.

The implementation of a modern full-stack architecture using HTML5, CSS3, JavaScript, Bootstrap, Flask, SQLite, and AI integration technologies provides a secure, responsive, and efficient user experience. The platform enables students to access customized learning pathways, AI-generated assessments, progress tracking tools, and downloadable reports while providing administrators with effective platform management capabilities.

Overall, LERNIX combines Generative AI, web technologies, educational methodologies, and software engineering principles to deliver an intelligent, scalable, and user-friendly learning management solution. The project highlights how AI-driven systems can improve educational accessibility, enhance learning experiences, and contribute to the future evolution of intelligent digital education platforms.